CSL 100/ MAL 100

Tutorial 9

Pointer and Dynamic array allocation

1) Write a program in C to find the largest element using Dynamic Memory Allocation

Input: total number of elements:: 5

Number 1: 5
Number 2: 7
Number 3: 2
Number 4: 9
Number 5: 8
The Largest element is : 9.00

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i, n;
    float *element; // Pointer to float type to store elements

    printf(" Input total number of elements (1 to 100): ");
    scanf("%d", &n);

    element = (float *)calloc(n, sizeof(float));
    if (element == NULL) {
        printf(" No memory is allocated.");
         exit(0);
    }
    printf("\n");
    for (i = 0; i < n; ++i) {
        printf(" Number %d: ", i + 1);
        scanf("%f", element + i);
    }
```

```c
for (i = 1; i < n; ++i) {
        if (*element < *(element + i)) {
            *element = *(element + i);
        }
    }
    return 0;
}
```

2) write a program that dynamically allocates memory for an array of 5 integers, fills it with values, then resizes the array to hold 10 integers. Fill the new elements with values and print the entire array

Input: Initial 5 integers:

1 2 3 4 5

After resizing to 10 integers:

1 2 3 4 5 6 7 8 9 10

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*)malloc(5 * sizeof(int));
    printf("Initial 5 integers:\n");
    for (int i = 0; i < 5; i++) {
        arr[i] = i + 1;
        printf("%d ", arr[i]);
    }

    arr = (int*)realloc(arr, 10 * sizeof(int));
    printf("\nAfter resizing to 10 integers:\n");
    for (int i = 5; i < 10; i++) {
        arr[i] = i + 1;
    }
    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

3) Reverse a String Using Pointers

Input: Input: Hello

Output: olleH

```c
#include <stdio.h>

void reverse(char *str) {
    char *end = str;
    while (*end) end++;
    end--;
    while (str < end) {
        char temp = *str;
        *str++ = *end;
        *end-- = temp;
    }
}

int main() {
    char str[] = "Hello";
    reverse(str);
    printf("Reversed string: %s\n", str);
    return 0;
}
```

Question

- **Problem Statement:** You are given an unsorted array of integers. Write a program in C to remove the duplicate elements in-place such that each element appears only once. The program should also return the new length of the modified array. You must modify the array in-place without using any extra space (like an auxiliary array or set or sorting is not allowed). And the sequence of the input should be the same. It should not change after removing the duplicate value.

  Example :Input: nums = {3, 1, 2, 3, 4, 1, 2, 5}

  n = 8

  Output: New length: 5

  Modified array: 3 1 2 4 5

  (Note: after removing duplicate value array size will reallocated as per number of unique number in the array)